

Module : Kotlin (M202)

TP N° 24

Année de Formation 2024/2025

Filière : Développement Mobile Groupe : DEVOAM101 - DEVOAM102 Niveau : 2ème année

Exercice 1 :

Création d'un CoroutineScope avec un Job :

1. Dans ce scope, lancez deux coroutines filles.
 - **Coroutine 1** : Cette coroutine va simuler une tâche qui échoue en levant une exception après un délai de 500 ms.
 - **Coroutine 2** : Cette coroutine va simuler une tâche qui devrait réussir après un délai de 1000 ms.
2. Observez ce qui se passe quand la première coroutine échoue. Vérifiez si la deuxième coroutine parvient à terminer.

Modification avec un SupervisorJob :

1. Remplacez le Job par un SupervisorJob dans le CoroutineScope.
2. Relancez les mêmes deux coroutines :
 - **Coroutine 1** échoue à nouveau après 500 ms.
 - **Coroutine 2** devrait continuer à s'exécuter et terminer avec succès malgré l'échec de la première.
3. Observez comment le SupervisorJob gère l'échec de l'une des coroutines et permet aux autres de continuer.

Exercice 2 :

1. Création d'une fonction avec launch :

- Créez une fonction qui lance deux coroutines utilisant **launch**. Chaque coroutine simule une tâche longue en utilisant **delay()**.

- Affichez un message au début et à la fin de chaque coroutine.
- 2. Création d'une fonction avec **async** et **await** :
 - Créez une fonction qui utilise **async** pour effectuer deux calculs asynchrones en parallèle. Utilisez **await** pour récupérer les résultats et afficher leur somme.
- 3. Utilisation de **runBlocking** pour tester les coroutines :
 - Utilisez **runBlocking** dans la fonction **main()** pour exécuter vos coroutines de manière synchrone. Lancez à la fois les coroutines avec **launch** et les calculs avec **async** dans ce scope.

Exercice 3 :

1. Création d'un **Flow** simple :
 - Créez une fonction qui génère un Flow d'entiers allant de 1 à 5. Chaque entier doit être émis avec un délai d'une seconde entre les émissions pour simuler une tâche asynchrone.
2. Collecte des données avec **collect** :
 - Utilisez l'opérateur **collect** pour récupérer et afficher chaque élément émis par le **Flow**.

Exercice 4 :

1. Création d'un **Flow** qui émet des nombres pairs :
 - Créez une fonction qui génère un Flow d'entiers pairs allant de 2 à 10. Chaque entier doit être émis avec un délai d'une seconde entre les émissions pour simuler une tâche asynchrone.
2. Collecte des données avec **collect** :
 - Utilisez l'opérateur **collect** pour récupérer et afficher chaque **nombre pair** émis par le Flow.